

Mind Bridge: Security Audit Report

System	Mind Bridge, confidential student wellness platform (University of San Agustin)
Report date	2026-10-02
Version audited	main at commit 5a75311
Audit type	White-box static review of the repository, plus <code>npm audit</code> and the existing emulator test suite
Classification	Internal. Contains no secrets, but describes weaknesses. Share on a need-to-know basis.

1. Executive summary

Mind Bridge has no application server. The React client talks directly to Firebase Auth and Cloud Firestore, so `firestore.rules` is the entire server-side security boundary. Most of this report is therefore about those rules and about what the client is trusted to do.

The foundations are sound. The rules are default-deny, self-signup cannot create a staff role, deactivation is enforced server-side, deletes are denied everywhere, and the rules have an automated test suite. The client has no XSS sinks and does not use cookie sessions.

The weaknesses cluster in three places, and they matter more here because the data is mental-health information, which Philippine law treats as sensitive personal information:

1. **Identity is weakly bound.** The `@usa.edu.ph` restriction and email verification are enforced only in the browser. Anyone can create an account with any address and get student access.
2. **Staff access is broad and unaudited.** Every approved counselor can read every student's check-ins and messages, and edit any field of any check-in.
3. **Students can write fields that should be server-controlled**, including the risk level of their own check-in, so a high-risk submission can be filed as "low".

There is also a compliance gap: the privacy policy promises erasure and blocking rights, but the system has no way to honor them.

Findings at a glance

ID	Severity	Finding
F-01	High	School-email restriction and email verification are not enforced server-side
F-02	High	Any approved staff member can read all student data and edit any check-in field; no audit trail

ID	Severity	Finding
F-03	High	Risk level and review flag are client-supplied; dashboards display the stored value
F-04	High	No erasure, export or retention mechanism despite documented data-subject rights
F-05	Medium	No security headers or Content Security Policy
F-06	Medium	Firestore rules do no input validation (types, lengths, enums, unknown fields)
F-07	Medium	Other client-writable fields: instantly confirmed appointments, slot un-booking, spoofed message sender
F-08	Medium	High-risk email alert is written but not deployed
F-09	Medium	Weak password policy and no MFA for staff and admin accounts
F-10	Medium	No Firebase App Check; API key restrictions unverified
F-11	Low	<code>.gitignore</code> does not cover <code>.env.production</code>
F-12	Low	Dependency audit: 4 high advisories, probably not reachable from the browser bundle
F-13	Low	Client route guard ignores <code>approved</code> ; role falls back to "student" on error
F-14	Low	Privacy policy inaccuracies; Google Fonts loaded from a third party

No critical findings. No committed secrets were found (see 8).

Scope and limits

Reviewed: `firestore.rules`, `client/src` (auth, API layer, validation, routing, all pages and features), `functions/index.js`, `client/vercel.json`, `.gitignore`, tracked env files, `package.json` files, git history for credential patterns, the `backend-tests/` suite.

Not reviewed, and not verifiable from the repository: the live Firebase console configuration (authorized domains, API key restrictions, email-enumeration protection, enabled providers, MFA, backups, IAM roles), the Vercel project settings, the production database contents, and the deployed copy of the rules. The user states the rules were published; this audit assumes the published rules match the repository file. No penetration test or live exploitation was performed. Items marked **verify** below depend on those console settings.

Severity scale

Level	Meaning
High	Exposes or lets an attacker alter sensitive student data, or defeats a safety control, with little effort
Medium	Weakens a defense or enables abuse that needs a precondition
Low	Hardening or hygiene; limited direct impact

2. Architecture and trust boundaries

```
Browser (React SPA, Vercel)
├─ Firebase Auth SDK → Firebase Auth (identity, ID tokens, 1 h)
├─ Firestore SDK → Cloud Firestore (users, assessments, appointments,
│                                     availability, messages)
│               ▲
│               └─ firestore.rules (the only server-side check)
└─ (not deployed) Cloud Function alertOnHighRisk → SMTP → staff inboxes
```

- **Trust the client for nothing.** Anything the browser sends can be forged with curl (see `docs/API.md` section 7). Every client-side check in this report is a usability aid, not a control.
- **Roles** live in `users/{uid}`: `student`, `counselor` (needs `approved: true`), `admin`. `active: false` locks an account out.
- **Public by design:** the Firebase web config (API key, auth domain, project id). These identify the project and are not secrets, but they make the REST endpoints reachable by anyone.

3. Authentication security

What works

- Passwords are handled entirely by Firebase Auth. The app never stores or logs them.
- Login errors are normalized: wrong password, unknown user and invalid credential all show the same message (`client/src/lib/errors.js`), which limits account enumeration through the UI.
- Sign-in throttling is provided by Firebase (`auth/too-many-requests` is handled).
- Self-signup can only create `role: "student"`, `approved: true`, `active: true`. A student cannot promote themselves: the rules reject changes to `role`, `approved`, `active` and the assignment fields (covered by `rules/users.test.js`).
- Counselors need admin approval before the rules grant any staff access.
- Deactivation is enforced by the rules (`isActive()`), not just the UI: a deactivated user's valid token is denied on every read and write.
- Session tokens are Firebase ID tokens sent as bearer headers. There is no cookie session.

F-01 (High): School-email restriction and verification are client-side only

Evidence. `Signup.jsx` and `Login.jsx` check `endsWith("@usa.edu.ph")` in the browser. The `users` create rule checks only `role`, `approved` and `active`. Sign-up stores `emailVerified: false` as a field the user

writes themselves, and nothing reads `request.auth.token.email_verified`.

Impact. - Anyone can call the Auth REST `signUp` endpoint with any email, then create a `users/{uid}` student profile, and receive full student access: submit check-ins, book appointments, message counselors. - Because the email is not verified, a person can register **someone else's** school address and act as that student. Counselors would see the wrong person's name and email on a mental-health case. This is an integrity and safety problem, not only a spam problem. - The `emailVerified` field in the profile is attacker-controlled and must not be relied on.

Recommendation. 1. Send a verification email at sign-up (`sendEmailVerification`) and block the app until verified. 2. Enforce in the rules, using the signed token, not profile fields:

```
function verifiedSchool() { return request.auth.token.email_verified == true && request.auth.token.email.matches('.*@usa[.]edu[.]ph$'); } // users create: add && verifiedSchool() // isActive(): add && verifiedSchool() for students (staff accounts are provisioned by an admin) Google sign-in tokens already carry email_verified: true. Staff and admin accounts provisioned out of band may use a different domain, so apply the domain clause to student creation only. 3. Treat the profile emailVerified field as informational, or remove it.
```

4. Firebase rules review

Reviewed file: `firestore.rules` (75 lines). It still carries a `// DRAFT` header comment even though it was published; update the comment so reviewers do not assume it is unreviewed.

Strengths

Control	Detail
Default deny	<code>match /{document=**} { allow read, write: if false; }</code> closes every unlisted collection
No deletes	Every collection denies delete, except a counselor deleting their own availability slot
Immutable messages	<code>messages</code> cannot be updated or deleted by anyone
Field-level guards	Users cannot change role, approval, active flag or assignment; students can change only <code>isBooked</code> on a slot, and only to cancel on an appointment
Profile required	<code>isActive()</code> needs an existing profile document, so a bare Auth account has no access
Tested	<code>backend-tests/</code> runs the real rules against the Firestore emulator (role by operation matrix, API contract, known-gap assertions)

F-02 (High): Staff access is unscoped and any check-in field is editable

Evidence. - `assessments` read: `studentId == auth.uid || isStaff()`. Any approved counselor reads **every** student's answers, notes and messages, not only their assigned students. The `assignedCounselorId` field exists but is never consulted by the rules. - `assessments` update: `allow update: if isStaff();` with no field restriction. A counselor, or a compromised counselor account, can

change `answers`, `total`, `riskLevel`, `studentId` or erase `counselorNotes`. - `messages` create lets any staff post into any student's thread. - There is no audit log of who read or changed what.

Impact. A single compromised or curious staff account exposes the whole student population's mental-health data and can rewrite clinical records undetectably. Under the Data Privacy Act this is the access-control and accountability weakness regulators look for first.

Recommendation. 1. Limit staff updates on assessments to the review fields: `allow update: if isStaff() && request.resource.data.diff(resource.data).affectedKeys().hasOnly(['status', 'counselorNotes', 'reviewedAt'])`; 2. Scope reads to assigned students, with admins and an explicit "triage" role as the exception: `get(/databases/(database)/documents/users/(resource.data.studentId)).data.assignedCounselorId == request.auth.uid`. This costs one extra `get()` per document and must be tested with list queries. 3. Add an audit trail. Without a server, the practical option is a Cloud Function (Blaze plan) that writes an append-only `auditLog` on assessment updates. Cloud Audit Logs for Firestore data access can also be enabled in the Google Cloud console. 4. Keep the number of admin accounts small and review them on a schedule.

F-03 (High): Risk level is client-supplied

Evidence. `submitResponse()` computes `riskLevel` and `flaggedForImmediateReview` in the browser and writes them. The create rule checks only `studentId` and `status == 'open'`. The known-gap test `rules/known-gaps.test.js` confirms a check-in with `total 21/21`, `riskLevel "low"` is accepted. The `alertOnHighRisk` function recomputes risk, but dashboards display the stored value.

Impact. A student in crisis who is also tampering, or a bug or stale client, produces a high-risk case that triage dashboards show as low. This defeats the platform's central safety function.

Recommendation. Do not trust the stored value. 1. Immediate, no backend needed: have the staff dashboard recompute risk from `total`, `maxScore` and `questionSummary` with `lib/scoring.js`, and show a warning when it differs from the stored `riskLevel`. 2. Better: enforce in the rules that `riskLevel` is consistent with `total` and `maxScore`, and that `flaggedForImmediateReview` is true whenever a crisis item is non-zero. 3. Best: let a Cloud Function write the authoritative risk fields and deny clients write access to them.

F-06 (Medium): No server-side input validation

The rules check *who* may write but not *what* is written. Today: - `status` on assessments and appointments accepts any string; the UI only offers a fixed set. - `counselorNotes`, `text` (messages), `answers`, `questionSummary` and `name` have no type or length limit server-side. The client uses `maxLength` (500, 2000, 80) in a few inputs, but REST calls bypass it. A user can store up to the 1 MiB document limit per write. - `answers` entries are not checked to be integers 0 to 3; `total` is not checked against them. - Users may add or change arbitrary fields on their **own** profile (for example `emergencyContact`, `email`, `name`), because only six fields are blacklisted. That includes `email`, which several screens treat as authoritative. - Create rules do not restrict the key set, so unknown fields are accepted.

Recommendation. Add validation helpers and use an allowlist for creates:

```
function validMessage(d) {
  return
  d.keys().hasOnly(['studentId', 'senderId', 'senderName', 'senderRole', 'text', 'timestamp'])
    && d.text is string && d.text.size() > 0 && d.text.size() <= 2000;
}
function validAssessment(d) {
  return d.status == 'open' && d.answers is list && d.answers.size() <= 50
    && d.total is int && d.maxScore is int && d.total >= 0 && d.total <= d.maxScore;
}
```

For updates use `hasOnly()` allowlists for self-service fields rather than a blacklist. Add a test per rule. Keep the client validators; they are good UX, just not a control.

F-07 (Medium): Other client-writable fields (known gaps 1, 2, 3, 5)

Documented and asserted by `rules/known-gaps.test.js`:

Gap	Effect	Fix
Student creates an appointment already <code>Confirmed</code>	Skips counselor review	Require <code>status == 'Pending Review'</code> on create
Student books any <code>slotId</code> or <code>counselorId</code> , even a booked slot	Double-booking; booking against an unrelated counselor	Check <code>get(slot).isBooked == false</code> and <code>counselorId == slot.counselorId</code> ; ideally use a transaction or function
Student can set any slot's <code>isBooked</code> back to <code>false</code>	Un-books another student's appointment	Allow only <code>false → true</code> for students; free slots via a function or staff
Student posts with <code>senderRole: "counselor"</code> and any <code>senderName</code>	Impersonation inside a confidential chat	Require <code>senderRole</code> to match the caller's <code>users.role</code> ; take <code>senderName</code> from the profile

When each is fixed, flip the matching `GAP:` test to `assertFails`.

5. Data protection and student privacy

Applicable framework

Mind Bridge processes names, institutional email, wellness check-in responses and counselor notes. Under the **Data Privacy Act of 2012 (RA 10173)** health information is *sensitive personal information*. The privacy policy also cites the Mental Health Act (RA 11036). This section is a technical assessment against those obligations, **not legal advice**; the university's Data Protection Officer should confirm the legal conclusions.

Controls in place

Principle	Status
Transparency	Privacy policy, terms and cookie policy exist and are linked; sign-up requires explicit consent
Purpose limitation	Policy states purposes; data is used only for triage and scheduling
Encryption	Firebase encrypts data in transit (TLS) and at rest by default, as the policy states
Minimization in alerts	The alert email omits answers and question text and links to the dashboard
Cookies	Only essential storage is used (<code>localStorage</code> for theme and consent); no analytics or ad trackers found
Confidential chat	Messages are append-only and limited to the student and staff

F-04 (High): Data-subject rights cannot be honored; no retention

Evidence. The policy promises access, rectification and the right to "suspend, withdraw, or order the blocking, removal, or destruction of your personal data". But: - Rules deny delete on `users`, `assessments`, `appointments` and `messages`; `messages` are also immutable. - There is no account-deletion, data-export or "request erasure" feature in the client (`deleteUser`, export and erase flows are absent). - No retention period is defined anywhere, so data is kept indefinitely. - Deactivation (`active: false`) blocks login but retains everything.

Impact. The university cannot meet a valid erasure or blocking request without ad hoc console edits, and indefinite retention of mental-health records is hard to justify.

Recommendation. 1. Define and publish a retention schedule (for example, close and anonymize check-ins a fixed number of years after the student leaves). 2. Document an admin-run erasure procedure using the Firebase console or Admin SDK, including deleting the Auth account, `users`, `assessments`, `appointments` and `messages` for that uid, and record each request. 3. Offer a self-service "request my data" and "request deletion" action that creates a ticket for the DPO. Full self-service deletion needs a Cloud Function. 4. Make sure backups and exports are covered by the same schedule.

Further compliance observations

- **Privacy policy accuracy (F-14).** It lists "student ID number" as collected, but the app does not collect one. It names no retention period, no Data Protection Officer contact or breach procedure, and it does not mention Google/Firebase as a processor. Update it to match reality.
- **Breach readiness.** Under RA 10173 and National Privacy Commission rules, breaches involving sensitive information must be reported to the NPC and affected individuals within **72 hours** of discovery. There is no breach-response document or contact path in the repo. Write one.
- **Registration and assessment.** Determine with the DPO whether the processing system must be registered with the NPC and whether a Privacy Impact Assessment is required. This report can serve as an input.
- **Processors and cross-border transfer.** Firebase (Google) stores the data and an SMTP provider would see student names in alert emails. Confirm the data-processing terms and hosting region

of the Firestore database (it is set at creation and cannot be changed).

- **"Confidential" is relative.** The chat is readable by every approved counselor and admin (F-02). Student-facing wording should say who can read it.
 - **Emergency contact data.** Stored on the profile, readable by all staff. Fine for the purpose, but it is third-party personal data and belongs in the retention and erasure scope.
-

6. Input validation, XSS and CSRF

Input validation

Client validators in `client/src/lib/validation.js` cover email, school domain, phone, password change, emergency contact and availability windows, with unit tests including edge cases. They are good UX but **client-only** (see F-06). The email regex accepts any `x@y.z` and the domain check is the only school-specific rule (see F-01).

XSS: no vulnerabilities found

- The code base has no `dangerouslySetInnerHTML`, `innerHTML`, `eval`, `document.write` or `window.open`. React escapes all interpolated text, including student names, notes and chat messages, which are the user-controlled strings staff see.
- The only dynamic `href` values are `tel:${phone}` links (`AdminPanel.jsx`, `CrisisResources.jsx`). The `tel:` prefix prevents a `javascript:` URI. The phone number is validated at input but, because of F-06, a student could store any string; the worst outcome is a malformed dial link. Constrain it in the rules or sanitize at render.
- The Cloud Function escapes `studentName` before placing it in the HTML email (`esc()`).
- **Residual risk:** with no CSP (F-05), a future XSS bug (for example from a dependency) could read the Firebase ID token from IndexedDB. CSP is the compensating control.

CSRF: not applicable to the current design

Authentication uses bearer tokens set in an `Authorization` header by the SDK, not cookies, so a cross-site request cannot carry credentials. There are no state-changing GET routes. The relevant neighbor risk is **clickjacking**, addressed in F-05. If cookie-based sessions are ever added (for example a Vercel serverless route for email), revisit CSRF tokens and `SameSite`.

F-05 (Medium): No security headers or CSP

Evidence. `client/vercel.json` contains only a catch-all rewrite. No response headers are set.

Impact. The app can be framed by another site (clickjacking on the check-in or booking flow), has no CSP to limit script injection, and sends full referrers.

Recommendation. Add headers in `client/vercel.json`. Start with the CSP in **report-only** mode, exercise sign-in (Google popup), Firestore and fonts, then enforce:

```
{
  "headers": [{
    "source": "/(.*)",
    "headers": [
      { "key": "X-Content-Type-Options", "value": "nosniff" },
      { "key": "X-Frame-Options", "value": "DENY" },
      { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
      { "key": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=()" },
      { "key": "Strict-Transport-Security", "value": "max-age=63072000; includeSubDomains" },
      { "key": "Content-Security-Policy-Report-Only", "value":
        "default-src 'self'; script-src 'self' https://apis.google.com; style-src 'self'
        'unsafe-inline' https://fonts.googleapis.com; font-src https://fonts.gstatic.com; img-src
        'self' data: https://*.googleusercontent.com; connect-src 'self' https://*.googleapis.com
        https://*.firebaseio.com; frame-src https://*.firebaseapp.com https://accounts.google.com;
        frame-ancestors 'none'; base-uri 'self'; object-src 'none'" }
    ]
  }]
}
```

This is a starting point, not a verified policy: Google sign-in loads extra origins, so test it before enforcing. Self-hosting the two Google Fonts (F-14) lets you drop the font origins.

7. Environment variable and secret security

Item	Result
client/.env.production	Tracked in git. Holds only <code>VITE_FIREBASE_API_KEY</code> , <code>AUTH_DOMAIN</code> , <code>PROJECT_ID</code> . These are public web-config values by design; it was committed deliberately (a24c70d) so production builds work. Acceptable.
Local .env	Ignored by .gitignore
Cloud Function secrets	<code>SMTP_URL</code> uses <code>defineSecret</code> (Secret Manager); <code>SMTP_FROM</code> and <code>APP_URL</code> are non-secret params. Correct pattern.
History scan	A search of recent revisions for private keys, <code>AKIA/sk_live</code> tokens, service-account files and committed SMTP URLs found none. The old <code>JWT_SECRET</code> appears only in README prose.
Vite exposure	Only <code>VITE_</code> -prefixed variables reach the bundle, and only the three Firebase values are used

F-11 (Low): .gitignore does not cover .env.production

The pattern `*.env` matches names ending in `.env`, not `.env.production`. That is why this file is tracked. If a real secret is ever put in an `.env.*` file it will be committed without warning. **Fix:** add `.env.*` and `!.env.example`, keep `client/.env.production` tracked via an explicit `!client/.env.production` if intended, and add a comment saying it must never hold secrets. Run a full-history scan once with a tool such as `gitLeaks`; this audit sampled recent history only.

F-10 (Medium): API key restrictions and App Check

The Firebase API key is public by design, so protection comes from **restrictions and App Check**, neither of which this audit could see. - **Verify**: in Google Cloud, restrict the browser key to HTTP referrers (the Vercel domain and `localhost`) and to the Identity Toolkit and Firestore APIs. - Enable **Firebase App Check** with reCAPTCHA so that only your app, not arbitrary scripts using the public key, can call Firestore and Auth. Today any script can. - **Verify** Auth settings: authorized domains list, email-enumeration protection enabled, unused sign-in providers disabled. - Set Google Cloud **budget alerts**; a leaked public key can still be used to run up usage.

8. Additional findings

F-08 (Medium): High-risk alert not deployed

`functions/index.js` emails approved staff when a high-risk check-in arrives, and it recomputes risk instead of trusting the client. It is **not deployed** (deployment needs the Blaze plan or an alternative route; the decision is deferred). Until then, a high-risk student is discovered only when a counselor opens the dashboard. Treat the platform as having **no active escalation path**, and make sure the counselor team knows to check the dashboard on a fixed schedule. When deployed, confirm the alert goes to a monitored shared mailbox, not individuals only, and that the SMTP provider is covered by a data-processing agreement. Note also: `functions/` has no lockfile, so builds are not reproducible and `npm audit` cannot run.

F-09 (Medium): Passwords and MFA

- Minimum password length is **6** (the Firebase default, mirrored in the client validator). That is below current guidance (NIST SP 800-63B suggests at least 8, longer preferred, with a breached-password check).
- There is no multi-factor authentication for **counselor and admin** accounts, which hold access to every student's records. Firebase MFA requires upgrading to Identity Platform (**verify** current requirements and cost).
- **Recommendation**: raise the client minimum to 10 or more characters, configure the Firebase password policy (length, character mix) in the console, and require MFA for admin and counselor accounts. Short of MFA, require institutional Google sign-in for staff.

F-12 (Low): Dependencies

`npm audit --omit=dev` in `client/` reports **4 high** advisories, all through `firebase` → `@firebase/firestore` → `@grpc/grpc-js`. That library is the **Node** gRPC transport; the browser Firestore build uses `WebChannel/fetch` and should not ship it, so the practical exposure is likely low. **Verify** with `npm ls @grpc/grpc-js` and by inspecting the built bundle. Do not run the suggested `npm audit fix --force`: it proposes downgrading Firebase to 9.14.0. Instead update to the latest Firebase 12.x and re-run the audit. `functions/` has no lockfile and could not be audited. Add `npm audit --omit=dev --audit-level=high` to CI.

F-13 (Low): Client role handling

- `ProtectedRoute` checks the role but not `approved`, so an **unapproved counselor** reaches the admin dashboard shell (empty, because the rules deny the data). Not exploitable, but it is confusing and should show a "pending approval" screen.
- `AuthContext` maps `counselor` to `admin` in the UI and, if the profile read fails, falls back to `"student"`. Fail-closed is correct, but the console logs the user's uid on a missing profile; drop that log in production.
- Route guards are UX only. All enforcement is in the rules, which is the right place.

F-14 (Low): Privacy policy and third-party assets

- The policy claims collection of a student ID number that is not collected; it omits retention, DPO contact, breach handling and the Firebase/Google processor. (See section 5.)
- `index.html` loads fonts from `fonts.googleapis.com` without a self-hosted fallback. This sends visitors' IP addresses to Google and forces CSP exceptions. Self-host the two font families.

9. Prioritized remediation plan

Priority	Action	Findings	Effort
Now (days)	Require verified <code>@usa.edu.ph</code> email in rules; send verification email	F-01	Small
Now	Restrict staff assessment updates to review fields	F-02	Small
Now	Recompute risk in the staff dashboard and flag mismatches	F-03	Small
Now	Add security headers (CSP report-only first)	F-05	Small
Now	Restrict API key; enable email-enumeration protection	F-10	Small
Soon (weeks)	Rules validation allowlists for all collections, with tests	F-06, F-07	Medium
Soon	Scope staff reads to assigned students	F-02	Medium
Soon	Write retention schedule, erasure procedure, breach response; fix privacy policy	F-04, F-14	Medium (process)
Soon	Raise password minimum; MFA for staff	F-09	Small to medium
Soon	Deploy the alert (or an interim manual triage schedule)	F-08	Medium
Later	App Check; audit log function; self-hosted fonts; gitleaks and <code>npm audit</code> in CI	F-10, F-02, F-14, F-12	Medium

Each rules change should be accompanied by a test in `backend-tests/rules/`, and flipping the matching GAP: assertion. Re-run `npm test` in `backend-tests/` before publishing the rules.

10. Positive observations

- Default-deny rules with explicit per-collection grants and no deletes.
- Server-enforced deactivation and counselor approval.
- No XSS sinks; no secrets in the repository; secrets in the function use Secret Manager.
- Automated rules, contract and function tests run in CI against the emulator.
- Known weaknesses are already documented and pinned by tests rather than hidden.
- Alert emails deliberately exclude answers and question text.

11. Conclusion

Mind Bridge's design is sensible for a serverless app, and the rule set is a good base. The highest-value work is to move the identity, role and risk-score checks that currently live in the browser into `firestore.rules`, narrow what staff can read and change, and close the data-subject-rights gap. The "Now" items in section 9 are each small and together close F-01 and F-02's worst case (arbitrary field edits) and mitigate F-03; scoped staff reads, rules-level risk enforcement and the F-04 process work follow. Once they are done, a follow-up review should test the live Firebase configuration, which this audit could not see.

Method: static code review and automated checks on 2026-10-02. This report assesses technical controls and is not a legal opinion or a penetration test.